

RelUI: Relational Behavioral Consistency Testing for Web UIs

CS 527 Research Project Report: Group 2

Ajmal Ali

Muneeb Hoda

University of Illinois at Urbana-Champaign

Urbana, Illinois, USA

{aali274,shoda6}@illinois.edu

Abstract

Source code: <https://github.com/MuneebHoda/ReUIs>

Modern Web UI bugs often hide between pages rather than inside one page. A confirmation screen can look normal while losing data after back navigation; a protected dashboard can render correctly while being reachable without login. This paper presents RelUI, a TypeScript and Playwright prototype for relational behavioral consistency testing of Web UIs. RelUI explores browser traces, canonicalizes states, infers a state-transition graph, and checks explicit workflow invariants over traces and endpoints. We evaluate the final artifact on four benchmark workflows, 20 subject variants, and 16 injected faults. The final run explored 454 replayable traces, discovered 230 canonical states and 467 transitions, detected all 16 injected faults, and reported zero clean false positives in 250.08 seconds. Lightweight structural and visual baselines detected only 5 and 0 faults, respectively, showing that workflow correctness needs relational oracles beyond local DOM or screenshot checks.

1 Introduction

Web applications increasingly encode critical behavior as multi-step workflows: login, checkout, account settings, and money transfer flows all depend on action history. These workflows fail in ways that are hard to detect with one-state checks. A page can contain the right controls and still violate an authentication precondition; two valid paths can reach the same confirmation URL but disagree about preserved user data.

Consider a small example from our money-transfer benchmark. The clean variant moves the user through amount entry, recipient selection, review, and confirmation; the receipt page shows the transferred amount. In the back-loses-transfer variant, going back from review to edit the recipient and then continuing forward reaches the same receipt URL but the displayed amount is missing. A structural crawler still finds the correct buttons on every page. A visual diff against the clean receipt looks essentially identical at the page level. A happy-path script that walks straight through still passes. The bug only surfaces when two paths that both end at Transfer Complete are compared against each other. RelUI flags this as a path-equivalence violation and produces both replay traces as evidence.

This kind of cross-path reasoning is the gap. Existing model-based testing constructs state graphs but assumes the engineer supplies the model [5]. Invariant-based Web testing checks predicates over individual states discovered by a crawler [2], but does not relate states reached through different paths. Vision-based GUI testing improves coverage by interpreting screens [6], but still reasons

4 apps	20 variants	16/16 faults	0 clean FPs
454 traces	230 states	467 edges	250.08s runtime

Figure 1: Final run scorecard generated from the upgraded artifact.

about screens, not about the relationships between them. None of these directly enforce constraints such as “every protected state must be preceded by an authentication action” or “two paths that mean the same thing must end in the same state.”

RelUI targets exactly this gap. It treats a Web UI as an explored transition system and checks five families of relational invariants over traces, graph edges, and endpoint states: authentication precondition, validation blocking, action-forbidden state, forbidden transition, and path equivalence. The implementation runs on Playwright [3] and emits JSON, CSV, HTML, screenshots, and LaTeX-ready tables.

We evaluated RelUI on four controlled benchmark applications (an authentication portal, a checkout wizard, a settings workflow, and a money-transfer flow), each shipping with one clean variant and four faulty variants for a total of 16 injected faults. Figure 1 summarizes the headline result. RelUI detected all 16 faults with zero false positives on the four clean variants. A structural baseline that compares visible control signatures detected 5 of 16, and a visual baseline based on per-pixel screenshot diffs detected 0 of 16. The full run took 250.08 seconds and built a graph of 230 canonical states and 467 transitions over 454 explored traces.

This paper makes three contributions:

- (1) A method for inferring a state-transition graph from automated Web UI exploration, with a canonicalization scheme that groups semantically equivalent pages while keeping behaviorally distinct ones separate.
- (2) A library of five relational invariant families that capture cross-path properties (authentication precondition, validation blocking, action-forbidden state, forbidden transition, and path equivalence), each driven by a small declarative configuration so the same checker works across applications.
- (3) An empirical study on 16 injected faults across four applications showing that relational invariants catch every fault while structural and visual baselines catch a third or



Figure 2: RelUI pipeline: browser exploration, semantic canonicalization, graph inference, relational checking, and evidence export.

fewer, with replayable counterexamples and a reproducible benchmark suite.

The rest of the paper is organized as follows. Section 2 sketches the pipeline. Section 3 formalizes each stage and presents the algorithm. Section 4 reports the experimental results. Section 5 positions our work against prior testing techniques. Section 6 discusses threats to validity, and Section 7 concludes.

2 Approach Overview

RelUI receives a subject URL, input profiles, seed traces, exploration bounds, and invariant definitions. It drives the UI in a real browser, snapshots reached states, merges equivalent states by semantic fingerprint, builds a labeled graph, and checks five invariant families.

The main design decision is to make the oracle relational. Instead of asking only whether a page looks different, RelUI asks whether a state is justified by the trace that reached it and whether alternative traces that implement the same task converge to equivalent endpoints.

3 Approach Details

3.1 Exploration and Canonicalization

RelUI combines deterministic seed traces with bounded action discovery. Seed traces guarantee coverage of security- and validation-relevant paths, while action discovery explores visible anchors, buttons, submit inputs, and role-button elements. Form actions are filled from named profiles such as `valid`, `admin`, and `invalid`.

Each reached page is snapshot as URL, title, visible text, visible controls, form values, and screenshot path. RelUI normalizes URLs, text, and control signatures, then hashes the semantic representation. Nodes with the same fingerprint are merged into a canonical state.

3.2 Relational Oracles

The final tool implements five invariant families: authentication/precondition, validation-blocking, action-forbidden-state, forbidden-transition, and path-equivalence. Listing 1 shows the new Transfer benchmark’s path-equivalence invariant. It checks that a normal transfer and a back/edit transfer path reach equivalent receipts after confirmation.

Listing 1: Transfer path-equivalence invariant from the final configuration.

```

1 {
2   id: "transfer-confirmation-paths-equivalent",
3   type: "path-equivalence",
4   severity: "medium",
5   description:
6     "Normal and back/edit transfer paths must converge to the same receipt state",
7   endpoint: { textIncludes: "Transfer Complete" },
8   candidateActionSelectors: ["[data-testid='confirm-transfer']"],
9   profiles: ["valid"],
10  minDistinctPaths: 2

```

Algorithm 1: RelUI exploration and checking

Input: Subject S , seed traces T_0 , profiles P , invariants I , bounds

Output: State graph G and violations V

```

1  $Q \leftarrow$  empty trace plus  $T_0$ ;  $G, V \leftarrow \emptyset$ ;
2 while  $Q$  has traces and trace bound is not exceeded do
3   pop trace  $t$ ; reset browser storage; navigate to  $S.url$ ;
4   replay  $t$  with Playwright and fill forms from  $P$ ;
5   snapshot reached pages and canonicalize them into state nodes;
6   add labeled transitions between consecutive canonical states;
7   if  $t$  is below depth bound then
8     discover visible enabled actions and enqueue unseen trace extensions;
9 foreach invariant  $i \in I$  do
10  evaluate  $i$  over traces, states, and graph edges; add counterexamples to  $V$ 
11 return  $G, V$ ;

```

3.3 Evidence and Exports

Every violation records invariant ID, type, severity, trace IDs, state IDs, replay steps, screenshots, and text deltas. The final exporter also computes app-level coverage and invariant-family coverage. This lets the report separate two questions: whether faults are detected, and what exploration/model size was needed to detect them.

4 Experimental Evaluation

4.1 Evaluation Plan

We evaluate three research questions:

- RQ1:** How effectively does RelUI detect injected workflow-level UI faults?
- RQ2:** Which faults are missed by structural and visual baselines?
- RQ3:** What exploration cost and model size does RelUI incur across benchmark apps?

The final benchmark suite contains Auth, Checkout, Settings, and Transfer workflows. Each workflow has one clean variant and four faulty variants, yielding 20 variants and 16 injected faults. The baselines are intentionally lightweight: a structural baseline flags unseen visible-control signatures, and a visual baseline compares screenshots from matching traces with pixel difference. They provide a diagnostic contrast, not a claim against full industrial testing suites.

4.2 Evaluation Results

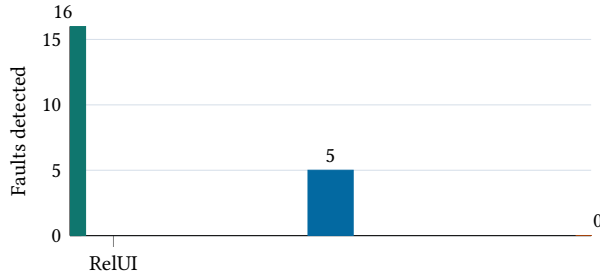
4.2.1 RQ1: Fault Detection. RelUI detected all 16 injected faults and produced zero relational violations on the four clean variants. The complete run discovered 230 canonical states, 467 transitions, and 454 replayable traces. Table 1 shows the per-app breakdown:

Table 1: App-level coverage and cost from the final run.

App	Variants	Faults	Detected	States	Trans.	Traces	Actions	Avg Len.	Max Len.	Runtime (s)
Auth	5	4	4/4	49	95	90	10	1.73	4	45.92
Checkout	5	4	4/4	69	150	142	11	2.25	6	91.77
Settings	5	4	4/4	45	95	100	9	1.2	3	40.29
Transfer	5	4	4/4	67	127	122	9	2.03	5	72.09

Table 2: Invariant family coverage. Configured oracles, faults assigned to each family, detected faults, and total violation instances reported.

Invariant Family	Configured	Faults	Detected	Violation Instances
action-forbidden-state	4	4	4/4	5
auth-precondition	3	3	3/3	7
path-equivalence	4	4	4/4	5
validation-blocking	6	5	5/5	42

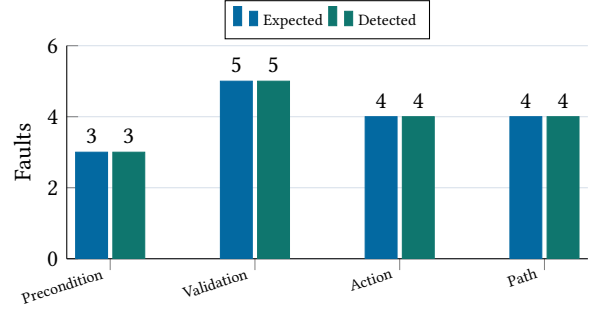
**Figure 3: Fault detection comparison. RelUI detects all 16 injected faults; structural and visual baselines detect 5 and 0.**

every app reports 4/4 detection, with Checkout the largest workflow by states, transitions, traces, and runtime, and Transfer adding a second multi-step, data-preservation-heavy benchmark.

Looking at which oracle family caught which fault is more informative than the aggregate. Table 2 summarizes the five configured families and the 59 total violation instances they produced. Validation-blocking accounts for 42 of those 59 instances over 5 faults, because every invalid input that should be rejected appears in many traces during exploration and each one fires the oracle independently. The other three families produced fewer instances but still detected every fault assigned to them: 7 instances over 3 authentication-precondition faults, 5 instances over 4 action-forbidden-state faults, and 5 instances over 4 path-equivalence faults. The forbidden-transition family was configured but had no fault assigned to it in this benchmark; it remains available for new subjects.

The zero-false-positive result on the four clean variants is the second half of effectiveness. The clean variants exercise the same paths and the same input profiles as the faulty ones, so a brittle oracle would have flagged them. The fact that none of the five families fired suggests the canonicalization granularity is in a reasonable band: tight enough to keep semantically equivalent states grouped together, loose enough to keep behaviorally distinct ones apart. This balance is what makes path-equivalence usable, since that oracle compares endpoint fingerprints directly.

4.2.2 RQ2: Baseline Misses. Figure 3 shows the headline comparison and Table 3 gives the per-fault breakdown. Structural checking

**Figure 4: Invariant-family contribution. Each configured oracle family detects all faults assigned to it.**

detects five faults: one Auth bypass, two Checkout validation failures, and two Transfer validation failures. It misses the other 11. The visual baseline detects 0 of the 16.

The interesting question is *why* the baselines miss what they miss. The five faults the structural baseline catches all change which controls appear on a page. F-AUTH-1 causes the dashboard to render in place of the login redirect, so the login form disappears. F-CHECKOUT-1 and F-CHECKOUT-3 let the workflow advance from a page that should have been blocked, so the next page contains controls it should not yet show. F-TRANSFER-1 and F-TRANSFER-2 do the same on the transfer flow. In each case the visible control set on at least one page differs from the matching clean state, which is exactly what the structural diff measures.

The other 11 faults leave the control structure of every visited page identical to the clean variant. F-AUTH-2 (logout-retains-access) renders the same logout button and the same dashboard layout as the clean app; the bug is that the dashboard remains reachable after the logout transition fires. F-SETTINGS-2 (cancel-persists-change) renders the same cancel button and the same summary page; the bug is that the saved name on the summary page is the edited value instead of the original. F-CHECKOUT-4, F-SETTINGS-1, F-SETTINGS-4, and F-TRANSFER-4 are path-equivalence faults: each individual page along each path is fine, but two paths that should converge produce different end states. None of these can be seen by comparing one state to one reference state. They become visible only when the relationship between two states or two paths is examined.

The visual baseline caught zero faults at the configured pixel-diff threshold (0.18 fraction of differing pixels). We chose the threshold to suppress noise from headless browser font rendering, which produced small but consistent diffs even between identical states. The injected faults are all logical, not cosmetic, so the visual diffs they produce stay below this threshold. Lowering the threshold would raise visual detections at the cost of a flood of false positives on the clean variants. This is the standard trade-off that makes screenshot regression testing brittle, and it is the reason a relational oracle is needed even when an evaluation team is willing to invest in pixel-level baselines.

4.2.3 RQ3: Exploration Cost. Figure 5 groups runtime and model size by workflow. Checkout is slowest at 91.77 seconds because it has the deepest workflow and largest transition set (150 transitions, average trace length 2.25, max 6). Transfer is the second largest at

Table 3: Fault coverage matrix. Structural and visual columns show whether the lightweight baselines detected the same faulty variant.

Fault	Workflow defect	RelUI oracle family	Structural	Visual
F-AUTH-1	Dashboard bypasses authentication	auth-precondition	Yes	–
F-AUTH-2	Logout retains protected access	action-forbidden-state	–	–
F-AUTH-3	User role reaches admin console	auth-precondition	–	–
F-AUTH-4	Invalid login reaches dashboard	validation-blocking	–	–
F-CHECKOUT-1	Invalid contact advances to payment	validation-blocking	Yes	–
F-CHECKOUT-2	Express review bypasses payment	action-forbidden-state	–	–
F-CHECKOUT-3	Negative quantity advances checkout	validation-blocking	Yes	–
F-CHECKOUT-4	Back path loses contact details	path-equivalence	–	–
F-SETTINGS-1	Quick profile save writes stale state	path-equivalence	–	–
F-SETTINGS-2	Cancel persists edited display name	action-forbidden-state	–	–
F-SETTINGS-3	Reset leaves edited display name	action-forbidden-state	–	–
F-SETTINGS-4	Quick notification save discards state	path-equivalence	–	–
F-TRANSFER-1	Invalid amount advances to recipient	validation-blocking	Yes	–
F-TRANSFER-2	Invalid recipient advances to review	validation-blocking	Yes	–
F-TRANSFER-3	Receipt reachable without confirm action	auth-precondition	–	–
F-TRANSFER-4	Back/edit path loses transfer amount	path-equivalence	–	–

72.10 seconds (127 transitions, average length 2.03, max 5). Auth runs in 45.92 seconds with shorter traces (average 1.73, max 4), and Settings is smallest at 40.29 seconds because most traces are one or two clicks long (average 1.20, max 3). The total across all 20 subjects is 250.08 seconds.

Two observations. First, runtime tracks transition count more closely than state count or trace count, because each new transition usually corresponds to one extra Playwright action (click or form fill) that has to be executed. Throughput is roughly 1.8 to 1.9 transitions per second across all four apps, which is consistent with the per-action latency of headless Chromium plus screenshot and DOM serialization overhead. The added Transfer benchmark scales the pipeline almost exactly as we would expect: it adds about 30% more transitions than the original three-app suite and adds about 30% more runtime.

Second, invariant checking is not the bottleneck. The five oracle families together iterate over the trace list and edge list once per invariant, and each check is a simple set membership or fingerprint comparison. The dominant cost is Playwright execution, not relational checking. This matters for scaling: more sophisticated invariants can be added without significantly increasing runtime, as long as the underlying graph is already built. Adding a new application costs roughly its share of new Playwright actions; the per-fault detection check is essentially free in comparison.

4.2.4 Representative Counterexamples. Beyond aggregate counts, RelUI produces a replay trace for every violation. Three counterexamples illustrate the kinds of bugs the relational oracles are meant to catch.

F-AUTH-1 (auth-bypass). The replay is a single step: open the dashboard from the home page. The end URL is `/auth/auth-bypass/dashboard` with the protected content rendered, and there is no login transition anywhere in the path. The `auth-precondition` oracle fires because the trace reaches a state matching the protected dashboard pattern without containing any of the required login selectors. The structural baseline catches this one too because the dashboard renders different controls than the expected login redirect.

F-AUTH-2 (logout-retains-access). The replay is four steps: log in, log out, then click the dashboard nav link. The end URL is `/auth/logout-retains-access/dashboard`, again with the protected content rendered. The `action-forbidden-state` oracle fires

because the trace contains a logout selector and a later state matches the protected dashboard. The control set is identical to the clean variant at every step, which is why the structural baseline misses it. This is a textbook case of a bug that requires comparing pre- and post-action state rather than inspecting any individual page.

F-TRANSFER-4 (back-loses-transfer). Figure 6 shows the path-equivalence fault from the Transfer benchmark. A normal transfer reaches a receipt page that preserves `Amount: $125`. A back/edit path that briefly returns to the recipient step before continuing forward reaches the same receipt URL after `confirm-transfer`, but the displayed amount is missing. The two endpoint fingerprints differ even though both paths satisfy the same task description. The page remains visually plausible at the receipt level, which is why the visual baseline misses it; the bug is structural in the workflow but not in any single rendered page.

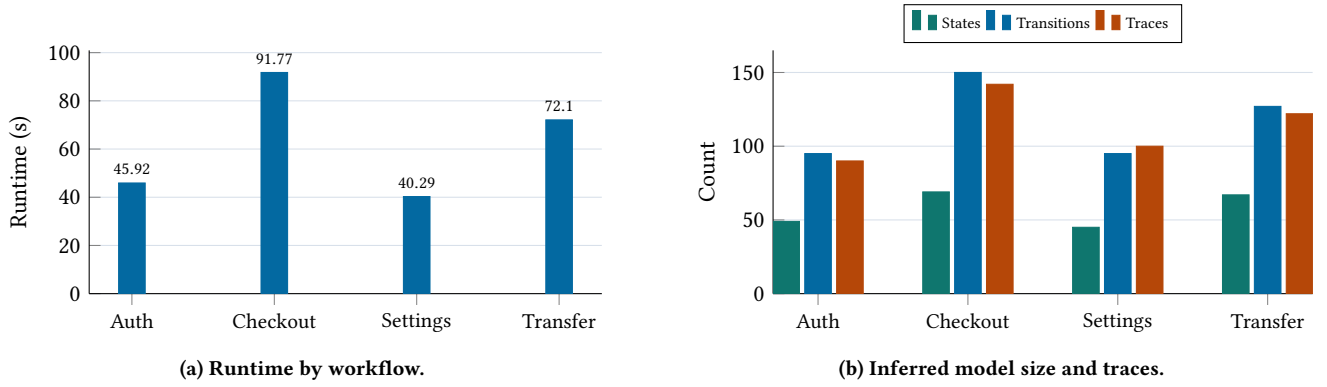


Figure 5: Runtime and model size after adding Transfer. The cost tracks workflow depth and branching more than the number of variants alone.

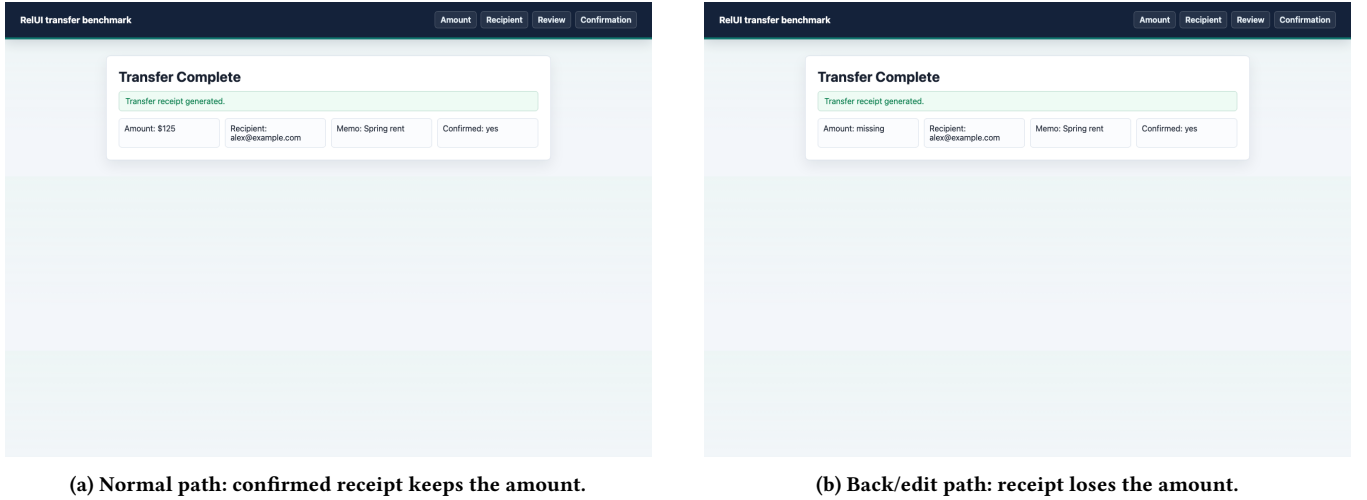


Figure 6: Transfer counterexample for F-TRANSFER-4. The two traces both end after confirm-transfer, but endpoint fingerprints differ: one receipt reports Amount: \$125, while the other reports Amount: missing.

5 Related Work

Model-based testing. Model-based testing derives tests from behavioral models [5]. The strength of these approaches is the ability to generate large, structured test suites; the weakness is that the model is an input, not an output, so the testing is only as good as the model the engineer writes. ReUI also uses a transition model, but infers it from browser traces and uses it mainly for relational oracle checking rather than for exhaustive model-derived generation.

Web state crawling and invariants. Crawljax explores Ajax applications and infers state-flow graphs [1]. Invariant-based Web testing checks discovered states against state and DOM constraints [2]. This line of work is the closest in spirit to ours and also infers a model from interaction rather than from a manual specification. The difference is in what the invariants describe. Existing invariants are mostly per-state predicates such as “this state should not contain a JavaScript error” or “this state should contain this element.” They do not relate two paths or compare a state before and after an action.

ReUI extends this lineage with five families that each quantify over multiple traces or multiple edges in the same graph.

GUI testing frameworks. GUITAR provides an extensible GUI testing framework across workflows and platforms [4]. Its strength is breadth across desktop and mobile UIs; its oracles are still primarily per-state. ReUI is narrower and Web-focused, but produces compact Playwright replay evidence around relational workflow defects.

Browser automation and LVLM GUI testing. Playwright supplies reliable browser automation and screenshots [3]. LVLM-based tools such as VETL use visual understanding for GUI exploration and input generation [6]. These approaches optimize for exploring more of the UI; they generally do not check relationships between explored paths.

Positioning. The four lines of work above differ from ReUI along one axis: the unit of reasoning. Model-based testing reasons over a model the engineer wrote. Invariant-based testing reasons over individual inferred states. GUI frameworks reason over

scripted sequences. Vision-based testing reasons over screens. RelUI reasons over *relationships* between paths and between pre- and post-action states, on top of an inferred model. The five invariant families we implemented are not properties of a single state; each one is a quantification over multiple traces or multiple edges in the same graph. This is what lets RelUI catch bugs such as “logout retains access” or “two paths diverge at the receipt,” which neither per-state invariants nor coverage-driven exploration can express.

6 Threats to Validity

Internal validity. The faults are controlled injections written by us, and each expected fault is mapped to an invariant in the configuration. This pairing improves auditability but means the results depend on the correctness of those mappings; an invariant that is too narrow could match its target fault by coincidence. We mitigated this by deriving each fault from a class of bug we observed in the wild (missing auth check, validation skip, state that should reset but does not, paths that should converge), and by implementing each fault as a single conditional branch in the benchmark server so the bug is small and localized. The canonicalization function is another internal threat: a too-aggressive canonicalization would merge distinct states and hide path-equivalence violations, while a too-strict canonicalization would split equivalent states and produce false positives. The clean-variant runs are the main check on this; they exercise the same paths as the faulty variants and produced zero violations, which suggests the granularity is in a usable band but does not prove it is correct on unseen UIs.

External validity. The four workflows are realistic surrogates, not production systems. They cover authentication, checkout, settings, and transfer behavior, but not all asynchronous, third-party, or highly dynamic UI patterns. We do not claim that 16-of-16 detection transfers to large production sites or to UIs generated by language models. What does transfer, we believe, is the categorical observation: the faults that escape structural and visual checks are the relational ones, and RelUI is built to catch that class. Confirming this on open-source applications and on generated UIs is future work.

Construct validity. The structural and visual baselines are intentionally lightweight diagnostic baselines. Their role is to show that local structure and appearance miss relational defects, not to replace a full comparison with mature commercial suites. Detection is also measured as “the relevant invariant fires at least once on the faulty variant,” which conflates “found a bug” with “produced an actionable signal.” A more careful construct would measure precision (whether the reported violations correspond to the injected fault rather than to a side effect) and developer effort (how long it takes to read the counterexample and confirm the bug). We report counterexample length as a partial proxy and we did not run a user study.

Reliability. RelUI resets browser storage before traces and uses deterministic input profiles, so reruns are stable up to Playwright timing variance. The trace budget (`maxTraces=80`, `maxDepth=4`, `maxActionsPerState=8`) bounds what the explorer reaches; bugs that live beyond this depth would not be discovered, and our scalability claims are calibrated to applications of the size used in this

benchmark. The artifact preserves graphs, traces, screenshots, CSV files, and HTML evidence to support reruns and manual inspection.

7 Conclusion

RelUI shows that Web UI correctness often depends on relationships across actions and paths. The final artifact evaluates four workflows, 20 variants, and 16 injected faults; it detects all faults with zero clean false positives, while local structural and visual baselines detect only 5 and 0. The faults the baselines miss are exactly the cross-path bugs that motivated this work: logout that does not actually log the user out, two checkout paths that diverge at the final screen, a cancel button that persists the change, a back/edit transfer that loses the amount on the receipt. The results support the core claim that workflow-level UI testing needs explicit relational oracles, not only page-local checks. Future work includes scaling the approach to larger open-source applications, evaluating against UIs generated by language models, and exploring learned canonicalization that adapts the abstraction granularity per application instead of using a fixed formula.

References

- [1] Ali Mesbah, Arie van Deursen, and Stefan Lenselink. 2012. Crawling Ajax-Based Web Applications Through Dynamic Analysis of User Interface State Changes. *ACM Transactions on the Web* 6, 1 (2012), 1–30. doi:10.1145/2109205.2109208
- [2] Ali Mesbah, Arie van Deursen, and Danny Roest. 2012. Invariant-Based Automated Testing of Modern Web Applications. *IEEE Transactions on Software Engineering* 38, 1 (2012), 35–53. doi:10.1109/TSE.2011.28
- [3] Microsoft Playwright. 2026. Playwright Browser Testing Documentation. <https://playwright.dev/docs/browsers>. Accessed 2026-05-05.
- [4] Bao N. Nguyen, Bryan Robbins, Ishan Banerjee, and Atif Memon. 2014. GUITAR: An Innovative Tool for Automated Testing of GUI-Driven Software. *Automated Software Engineering* 21, 1 (2014), 65–105. doi:10.1007/s10515-013-0128-9
- [5] Mark Utting, Alexander Pretschner, and Bruno Legeard. 2012. A Taxonomy of Model-Based Testing Approaches. *Software Testing, Verification and Reliability* 22, 5 (2012), 297–312. doi:10.1002/stvr.456
- [6] Siyi Wang, Sinan Wang, Yujia Fan, Xiaolei Li, and Yepang Liu. 2024. Leveraging Large Vision Language Model for Better Automatic Web GUI Testing. arXiv:2410.12157 [cs.SE] doi:10.48550/arXiv.2410.12157